lab3

April 27, 2026

To calculate the prices p_1 and p_2, we use the Leontief price model system:

$$p_j = \sum_{i=1}^{n} a_{ij} p_i + v_j$$

Where: - $a_{ij} = \frac{x_{ij}}{X_j}$ are the direct input coefficients. - v_j is the value added per unit of output in sector j. - X_j is the total output of sector j.

[ ]:
```
import numpy as np

# x11: Industry to Industry, x12: Industry to Agriculture
# x21: Agriculture to Industry, x22: Agriculture to Agriculture
x11, x12 = 600, 450
x21, x22 = 450, 400

# Final consumption (Y)
y1, y2 = 1200, 700

# 2. Calculate Total Output (X)
X1 = x11 + x12 + y1
X2 = x21 + x22 + y2

print(f"Total Industrial Output (X1): {X1} million UAH")
print(f"Total Agricultural Output (X2): {X2} million UAH")

# 3. Calculate the Direct Input Coefficients Matrix (A)
# a_ij = x_ij / X_j
a11 = x11 / X1
a21 = x21 / X1
a12 = x12 / X2
a22 = x22 / X2

# The matrix A is formed by columns a_ij
A = np.array([
    [a11, a21],
    [a12, a22]
])

# 4. Value Added Vector (V)
```

```
V = np.array([0.3, 0.5])

# 5. Solve the linear system: P = A^T * P + V  => (I - A^T) * P = V
# Create Identity Matrix I
I = np.eye(2)

# Matrix for the system (I - A.T)
M = I - A

# Solve the system of linear equations M * P = V
P = np.linalg.solve(M, V)

print("-" * 40)
print(f"Price for Industrial products (p1): {P[0]:.4f}")
print(f"Price for Agricultural products (p2): {P[1]:.4f}")
```

```
Total Industrial Output (X1): 2250 million UAH
Total Agricultural Output (X2): 1550 million UAH
----------------------------------------
Price for Industrial products (p1): 0.6637
Price for Agricultural products (p2): 0.9336
```

1 Matrix Productivity Analysis and Leontief Inverse Calculation

We are analyzing a Direct Input Matrix A representing a three-sector economy:

$$A = \begin{pmatrix} 0.4 & 0.4 & 0.2 \\ 0.2 & 0.5 & 0.4 \\ 0.1 & 0.1 & 0.2 \end{pmatrix}$$

Key Objectives: - Determine the stability and productivity of the economy via Frobenius eigenvalues. - Calculate the Leontief Inverse (Total Expenditures Matrix B). - Test the Neumann Series convergence ($\sum A^N$) which represents the layers of indirect costs. - Forecast the required Total Output (X) for a specific consumer demand (y).

[ ]:
```
import numpy as np
from numpy.linalg import eig, inv, matrix_power

A = np.array([
    [0.4, 0.4, 0.2],
    [0.2, 0.5, 0.4],
    [0.1, 0.1, 0.2]
])

# Vector y: Final demand (what consumers want to buy)
y = np.array([100, 70, 80])

print("--- Step 1: Input Matrix A ---")
print(A)
```

```
# eigenvalues solve the equation det(A -  I) = 0
eigenvalues = eig(A)[0]
# poly() returns the coefficients of the characteristic polynomial
char_poly = np.poly(A)

print(f"\nEigenvalues: {eigenvalues}")
print(f"Characteristic Polynomial Coefficients: {char_poly}")

# The Frobenius number is the largest real eigenvalue.
# It determines if the system is productive.
lambda_max = max(eigenvalues.real)
print(f"\nFrobenius Number ( _max): {lambda_max:.4f}")

# Finding Right Frobenius Vector (structure of output)
vals, vecs = eig(A)
right_frob = vecs[:, np.argmax(vals.real)].real

# Finding Left Frobenius Vector (often related to 'intrinsic' prices)
# We find eigenvalues of the transposed matrix A.T
vals_l, vecs_l = eig(A.T)
left_frob = vecs_l[:, np.argmax(vals_l.real)].real

print(f"Right Frobenius Vector: {right_frob}")
print(f"Left Frobenius Vector: {left_frob}")

# If  _max < 1, the economy produces a surplus.
if lambda_max < 1:
    print("\nVERDICT: The matrix is PRODUCTIVE ( _max < 1).")
else:
    print("\nVERDICT: The matrix is NON-PRODUCTIVE. The system consumes more␣
 ↪than it produces.")

# B = (I - A)^-1. Elements b_ij show how much of sector i
# is needed to produce 1 unit of FINAL product for sector j.
I = np.eye(3)
B = inv(I - A)
print("\n--- Total Expenditures Matrix (B) ---")
print(B)

# We check if E + A + A^2 + ... + A^N converges to B.
# This represents adding up direct costs + indirect costs + 2nd order costs...
S_n = np.zeros((3, 3))
n = 0
precision = 0.01
diff = 1.0
```

```python
while diff >= precision:
    S_n += matrix_power(A, n)
    diff = np.max(np.abs(B - S_n))
    n += 1

print(f"\n--- Convergence Analysis ---")
print(f"The series converged to B at step N = {n-1}")
print(f"Max difference at convergence: {diff:.5f}")

# X = B * y. Calculates the gross production required.
X = B.dot(y)
print(f"\n--- Required Total Output (X) ---")
for i, val in enumerate(X):
    print(f"Sector {i+1}: {val:.2f} million UAH")
```

```
--- Step 1: Input Matrix A ---
[[0.4 0.4 0.2]
 [0.2 0.5 0.4]
 [0.1 0.1 0.2]]

Eigenvalues: [0.83971676+0.j         0.13014162+0.06707427j
0.13014162-0.06707427j]
Characteristic Polynomial Coefficients: [ 1.    -1.1    0.24  -0.018]

Frobenius Number ( _max): 0.8397
Right Frobenius Vector: [0.7089431  0.67144485 0.21578112]
Left Frobenius Vector: [0.44397979 0.69076468 0.57072419]

VERDICT: The matrix is PRODUCTIVE ( _max < 1).

--- Total Expenditures Matrix (B) ---
[[2.95081967 2.78688525 2.13114754]
 [1.63934426 3.7704918  2.29508197]
 [0.57377049 0.81967213 1.80327869]]

--- Convergence Analysis ---
The series converged to B at step N = 33
Max difference at convergence: 0.00892

--- Required Total Output (X) ---
Sector 1: 660.66 million UAH
Sector 2: 611.48 million UAH
Sector 3: 259.02 million UAH
```